

5-Mark Question

Q. Explain any five features of Java.

Answer

1. **Platform Independent** – Java programs run on any system with JVM.
2. **Object-Oriented** – Uses classes and objects.
3. **Secure** – Does not use pointers directly.
4. **Robust** – Handles memory and exceptions effectively.
5. **Multithreaded** – Executes multiple tasks simultaneously.

These features make Java reliable and portable.

Q. What is Object-Oriented Programming? Explain its main principles.

Answer

Object-Oriented Programming (OOP) is a programming paradigm based on objects and classes. It helps in organizing large programs and provides security and reusability.

Main principles of OOP are:

1. **Encapsulation** – Binding data and methods together.
2. **Inheritance** – Acquiring properties from another class.
3. **Polymorphism** – One method performing multiple tasks.
4. **Abstraction** – Hiding implementation details.

OOP improves:

- Reusability
- Security
- Maintainability

Languages supporting OOP are Java, C++, and Python.

1. `psvm : public static void main(String[] args)`

This is the **entry point** of any Java application. When you run a program, the Java Virtual Machine (JVM) actively looks for this exact line to start executing your code.

- `public` : An access modifier. It means this method is visible to anyone, which is necessary so the JVM can find and run it from outside the class.
- `static` : Means the method belongs to the class itself, not to a specific object. The JVM can run this method without needing to create an instance (object) of your class first.
- `void` : The return type. It means this method does not return any data back to the JVM when it finishes running.
- `main` : The mandatory name of the method. The JVM is hardcoded to look for exactly this name.
- `String[] args` : The parameter that accepts an array of Strings. This is exactly where the command-line arguments you just learned about are stored.

2. `sout : System.out.println()`

This is the standard command used to print text or data to the console/terminal.

- `System` : A built-in Java class that holds underlying system resources and utilities.
- `out` : A static property inside the `System` class. It represents the "Standard Output Stream" (which is usually your monitor/terminal).
- `println()` : A method that belongs to `out` . It prints whatever data you put inside the parentheses, and the "ln" part stands for "line," meaning it automatically moves the cursor to the next line after printing. *(If you use just `print()` without the "ln", it stays on the same line).*

(A) Primitive Data Types

These are the most basic data types, predefined by Java. There are **8 primitive types**:

Data Type	Size	Example	Description
byte	8-bit	10	Small integer values (-128 to 127)
short	16-bit	1000	Larger integer values (-32,768 to 32,767)
int	32-bit	50000	Commonly used for whole numbers
long	64-bit	100000L	Very large integers
float	32-bit	3.14f	Decimal numbers (single precision)
double	64-bit	3.14159	Decimal numbers (double precision)
char	16-bit	'A'	Single character (Unicode)
boolean	1-bit	true/false	Logical values

Data Types



(B) Non-Primitive (Reference) Data Types

These are created by the programmer or provided by Java.

They store **references** (addresses) to objects. Examples:

- **String** → "Hello"
- **Arrays** → `int[] arr = {1, 2, 3};`
- **Classes & Objects** → `Student s = new Student();`
- **Interfaces, Enums, Collections (List, Map, etc.)**

Type Conversion

(A) Implicit Conversion (Type Promotion)

- Smaller data type → Larger data type automatically.

```
int x = 10;
```

```
double y = x;    // int → double
```

(B) Explicit Conversion (Type Casting)

- Programmer-defined conversion.

```
double d = 9.78;
```

```
int i = (int) d;    // double → int
```

Summary

Method	Type	Case Sensitive	Compares Values	Compares References
==	Reference check	N/A	✗	✓
equals()	Method	✓	✓	✗
equalsIgnoreCase()	Method	✗	✓	✗
compareTo()	Method	✓	✓ (lexicographic)	✗

]

String | StringBuffer | StringBuilder

Aspect	String	StringBuffer	StringBuilder
Mutability	✗ Immutable	✓ Mutable	✓ Mutable
Thread-safe	✓ Yes (immutable)	✓ Yes (synchronized)	✗ No
Methods synchronized	✗ No	✓ Yes	✗ No
Modification behavior	Creates new object	Modifies same object	Modifies same object
Performance (modifications)	✗ Slow	⚠ Medium	✓ Fast
Memory usage	✗ More (many objects)	✓ Less	✓ Less
Concurrency overhead	None	High (synchronization)	None
Introduced in Java	Java 1.0	Java 1.0	Java 1.5
Use case	Fixed strings, constants	Multi-threaded environment	Single-threaded environment
Example methods	concat()	append(), insert()	append(), insert()

CONSTRUCTORSS



Constructors – Some properties

1. A constructor **initializes an object** immediately upon creation.
2. Constructor in Java is a **method**.
3. This method has the **same name** as the class in which it resides.
4. Once defined, the constructor is **automatically called** immediately after object is created.
5. Constructor is a method which has **no return type**.
6. In fact, the implicit return type of a class constructor is the class type itself.
7. Constructor initialize the internal state of an object.

Default and parameterised !

```
Java
class Car {
    String brand;
    int year;

    // 1. Default Constructor (No-Argument)
    // Used when we don't have specific details at the time of creation
    Car() {
        this.brand = "Unknown Brand";
        this.year = 0;
    }

    // 2. Parameterized Constructor
    // Used when we know the exact details at the time of creation
    Car(String brand, int year) {
        this.brand = brand;
        this.year = year;
    }

    // A simple method to print the car's details
    void display() {
        System.out.println("Brand: " + brand + ", Year: " + year);
    }
}

public class Main {
    public static void main(String[] args) {

        // Creating an object using the Default Constructor
        // We pass no arguments inside the parentheses ()
        Car car1 = new Car();

        // Creating an object using the Parameterized Constructor
        // We pass the specific details inside the parentheses
        Car car2 = new Car("Toyota", 2023);

        // Let's see the output
        System.out.println("Output for car1 (Default):");
        car1.display();

        System.out.println("\nOutput for car2 (Parameterized):");
        car2.display();
    }
}
```

THIS KEYWORD



this keyword concept

1. `this` is used to reduce name-space collisions.
2. Sometimes a method will need to refer to the object that invoked it.
3. To allow this Java defines `this` keyword.
4. `this` can be used inside any method to refer to the current object.
5. That is, `this` is always a reference to the object on which the method is invoked.

THIS KEYWORD COMES IN USE WHENN THERE ARE MULTIPLE OBJECTS IN THE CONSTRUCTOR

HELPS IN CONSTRUCTOR CHAINIING AND IS USEFUL WHEN THERE IS CONSTRUCTOR OVERLOADING

```
// Helper class for Rule 5
class Engine {
    Engine(Car car) {
        System.out.println("1. Engine installed in: " + car.model);
    }
}

// Helper class for Rule 4
class Mechanic {
    void inspect(Car car) {
        System.out.println("2. Mechanic is inspecting: " + car.model);
    }
}

class Car {
    String model;
    int speed;

    // --- RULE 2: Invoke another constructor ---
    // The no-arg constructor uses this() to call the parameterized constructor
    Car() {
        this("Generic Car");
    }

    // --- RULE 1: Resolve name-space collisions ---
    Car(String model) {
        this.model = model;
        this.speed = 0;

        // --- RULE 5: Pass object as an argument in a constructor ---
        // We pass the current Car object to the Engine constructor
        Engine myEngine = new Engine(this);
    }

    // --- RULE 6: Return the current class instance ---
    // By returning 'this', we can chain methods together (Builder Pattern)
    Car setSpeed(int speed) {
        this.speed = speed;
        System.out.println("3. Speed set to: " + this.speed);
        return this;
    }

    // --- RULE 3: Invoke current class methods ---
    void start() {
        // Using this.methodName() to call another method in the same class
        this.checkFuel();
        System.out.println("5. " + model + " is now running at " + speed + " km/h");
    }

    void checkFuel() {
        System.out.println("4. Fuel levels checked and looking good.");
    }

    // --- RULE 4: Pass object as an argument in a method ---
    void takeToShop(Mechanic mechanic) {
        // We pass the current Car object to the mechanic's inspect method
        mechanic.inspect(this);
    }
}

public class Main {
    public static void main(String[] args) {
        Mechanic mike = new Mechanic();

        // Let's trigger all the rules!

        // Triggers Rule 2, which triggers Rule 1 & Rule 5
        Car myCar = new Car("Mustang");

        System.out.println("-----");

        // Triggers Rule 4
        myCar.takeToShop(mike);

        System.out.println("-----");

        // Triggers Rule 6 (method chaining) and Rule 3
        myCar.setSpeed(120).start();
    }
}
```

Types of Polymorphism

1. Compile-Time Polymorphism (Static) (Method Overloading)

Same method name, but different parameter list. The compiler decides which method to call based on the method signature.

```
class Calculator {  
    int add(int a, int b) {  
        return a + b;  
    }  
  
    double add(double a, double b) {  
        return a + b;  
    }  
  
    int add(int a, int b, int c) {  
        return a + b + c;  
    }  
}
```

Types of Polymorphism

2. Run-Time Polymorphism (Method Overriding)

A subclass provides a specific implementation of a method that is already defined in its superclass.

```
class Animal {  
    void sound() {  
        System.out.println("Animal makes sound");  
    }  
}  
  
class Dog extends Animal {  
    void sound() {  
        System.out.println("Dog barks");  
    }  
}  
  
Animal a = new Dog();  
a.sound(); // Output: Dog barks (runtime decision)
```



Even though a is of type Animal, **at runtime**, the Dog version of sound() is called.

If you make an object of Dog like Dog a = new Dog(), then it will be just method overriding no run time polymorphism as compiler already know which to run at compile time.

Static Variable

Static Variable (Class Variable)

1. **Belongs to the class, not to objects.**
2. **Only one copy is created** for the entire class, no matter how many objects are created.
3. Initialized only **once** when the class is **first loaded** into memory.
4. **Shared among all instances** (objects) of the class.
5. Can be accessed using:
 1. `ClassName.variableName` (recommended)
 2. `objectName.variableName` (not recommended, but allowed)
6. Often used for **common properties** (e.g., college name, company name, counter, etc.)
7. Can be modified, but not redeclared once defined.
8. We cannot declare a static variable inside a method (function) in Java.

Static Method

1. **Belongs to the class**, not to any specific object.
2. Can be called **without creating an object** using `ClassName.methodName()`.
3. **Can only access static data members** directly (cannot access non-static variables or methods directly).
4. Cannot use `this` or `super` keyword inside a static method (since it doesn't refer to any instance).
5. Commonly used for **utility or helper methods** (e.g., `Math.max()`, `Collections.sort()`).
6. If you try to access non-static members, you'll get a **compile-time error**.

SUPER keyword

Use Case	Syntax Example
Call parent constructor	<code>super();</code>
Call parent method	<code>super.methodName();</code>
Access parent variable	<code>super.variableName;</code>

Java



```
// PARENT CLASS (Papa)
class Vehicle {
    int maxSpeed = 120; // Parent variable

    // Parent Constructor
    Vehicle() {
        System.out.println("1. Vehicle (Parent) is created.");
    }

    // Parent Method
    void sound() {
        System.out.println("3. Vehicle makes a generic vroom sound.");
    }
}

// CHILD CLASS (Beta)
class Car extends Vehicle {
    int maxSpeed = 200; // Child variable (hides parent's variable)

    // Child Constructor
    Car() {
        // USE #3: Invoking Parent Constructor
        // This must be the very first line!
        super();
        System.out.println("2. Car (Child) is created.");
    }

    // Child Method (Overrides parent method)
    void sound() {
        // USE #2: Invoking Parent Method
        super.sound();
        System.out.println("4. Car makes an aggressive V8 engine sound!");
    }

    void displaySpeeds() {
        System.out.println("My Speed: " + this.maxSpeed); // Points to Car's

        // USE #1: Accessing Parent Variable
        System.out.println("Parent Speed: " + super.maxSpeed); // Points to Vehi
    }
}

public class Main {
    public static void main(String[] args) {
        Car myCar = new Car();
        System.out.println("-----");
        myCar.sound();
        System.out.println("-----");
        myCar.displaySpeeds();
    }
}
```

Final keyword :

Basically used with

Instance var

Local var

Static var

Methods and classes

Methods cant be overridden and classes cant be inherited

ABSTRACT VS INTERFACE

- **Abstraction (Abstract Class):** Abstraction is the process of hiding the background details and showing only the essential features to the user. An `abstract` class is a restricted class that cannot be used to create objects directly (you cannot use `new AbstractClass()`). It exists solely to be inherited by other classes.

- **Interface:** An interface is a completely abstract "blueprint" or "contract."

Example A: Abstract Class

Java



```
// You cannot create an object of 'Shape'
abstract class Shape {
    // Abstract method (No body, forces child to implement)
    abstract void draw();

    // Normal method (Has a body, shared by all children)
    void displayColor() {
        System.out.println("Color is Red.");
    }
}

class Circle extends Shape {
    // Child MUST implement the abstract method
    void draw() {
        System.out.println("Drawing a Circle ○");
    }
}
```

Example B: Interface

Java



```
// The Contract
interface Playable {
    // Methods in interfaces are inherently 'public abstract'
    void play();
    void pause();
}

// The Class signing the contract
class AudioPlayer implements Playable {
    // Must provide the exact implementation
    public void play() {
        System.out.println("▶ Playing the song...");
    }
    public void pause() {
        System.out.println("⏸ Song paused.");
    }
}
```

DIAMOND PROBLEM DEKH JA BHAIIIIIII!!!!!!!

MID SEM DONEEEEEEEEE!!!